TECHNICAL REPORT · TEAM PROJECT · M2 STATISTICS, SORBONNE UNIVERSITY · DECEMBER 2022

Online Convex Optimization: Linear SVM on MNIST

First-order, mirror-descent and second-order methods implemented from scratch in NumPy, and a reproduction of SVRG (Johnson & Zhang, NeurIPS 2013).

Code: github.com/6inzia/online-convex-optimization-svrg

1 Problem

We classify handwritten digits with a linear Support Vector Machine, in a one-vs-rest setting: **digit 0 (label +1) vs all other digits (label −1)**. Each 28×28 image is a vector $a_i \in \mathbb{R}^{785}$ (784 pixels rescaled to $[0,1]$ plus an intercept). We minimize the regularized hinge loss

$$\min_{x\in\mathbb{R}^{785}} \; f(x) = \frac{1}{n}\sum_{i=1}^{n} \max\{0,\; 1 - b_i\, x^\top a_i\} + \frac{\lambda}{2}\|x\|^2.$$

For $\lambda = 0$ the problem is convex; for $\lambda > 0$ it is strictly convex. Constrained variants project the iterates onto the ℓ_1 ball of radius z, $\mathcal{B}_1(z) = \{x : \|x\|_1 \le z\}$.

2 Algorithms

Family	Algorithm	Update	Step size
Gradient descent	GD / Projected GD	$x_{t+1} = \Pi_{\mathcal{B}_1(z)}\big(x_t - \eta_t \nabla f(x_t)\big)$	constant, $1/\sqrt{t}$, $1/(\lambda t)$
Stochastic	SGD / Projected SGD	same with the gradient of one random sample, averaged iterates	constant, $1/(\lambda t)$
Regularized Follow The Leader	Stochastic Mirror Descent	lazy projection of the accumulated gradients	$1/\sqrt{t}$
	Exponentiated Gradient ±	multiplicative weights on $2d$ coordinates, $x = z(w^+ - w^-)$	$1/\sqrt{t}$
	AdaGrad	per-coordinate step $\propto 1/\sqrt{\sum_s g_s^2}$, weighted ℓ_1 projection	adaptive
Second order	Online Newton Step	$A_t = A_{t-1} + g_t g_t^\top,\; x_{t+1} = \Pi^{A_t}\big(x_t - \frac{1}{\gamma}A_t^{-1} g_t\big)$	$1/\gamma$
Variance reduction	SVRG	$x_{t+1} = x_t - \eta\big(\nabla f_i(x_t) - \nabla f_i(\tilde{w}) + \nabla f(\tilde{w})\big)$	constant

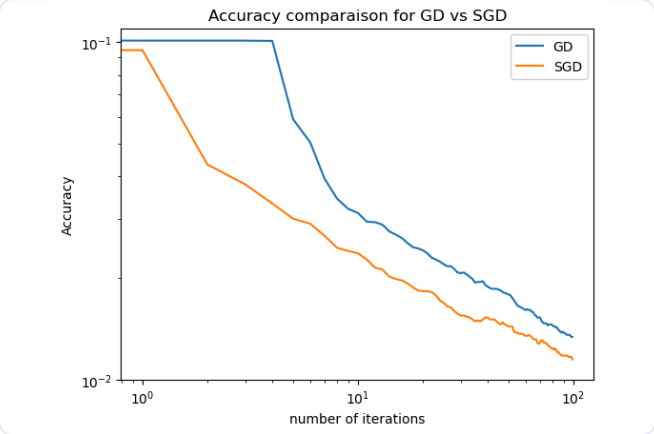
The projections onto the simplex, the ℓ_1 ball and the weighted ℓ_1 ball are implemented as well (sort-based algorithm, $O(d \log d)$).

3 Experimental setup

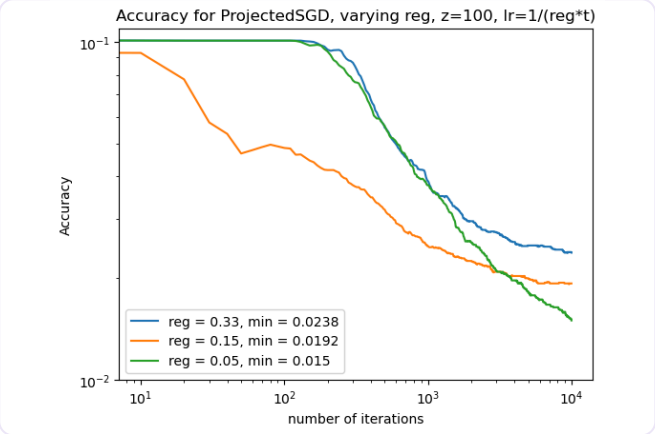
- **Data**: 10,000 MNIST training images (1/6 of the data set, shuffled) to keep computation time reasonable.
- **Metric**: misclassification rate on the training set (lower is better). Always predicting −1 gives about 10%.
- **Stochastic methods**: the averaged iterate $\bar{x}_t = \frac{1}{t}\sum_{s\le t} x_s$ is evaluated.
- **Comparisons at equal computational budget**: when an algorithm is k times faster per iteration, it gets k times more iterations.

4 Results

Gradient descent vs stochastic gradient descent

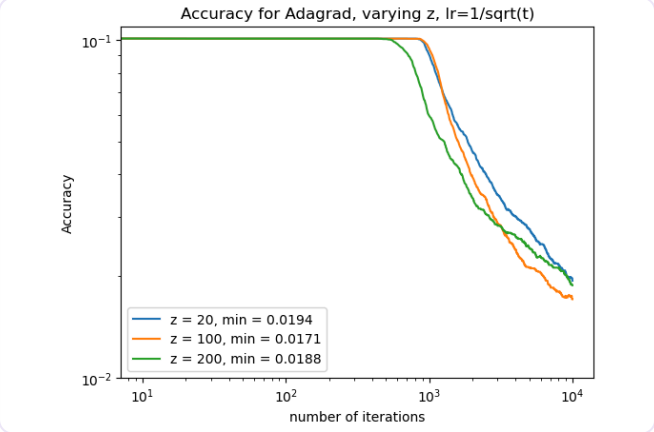


GD vs SGD at equal computation time: SGD reaches a lower error much faster.

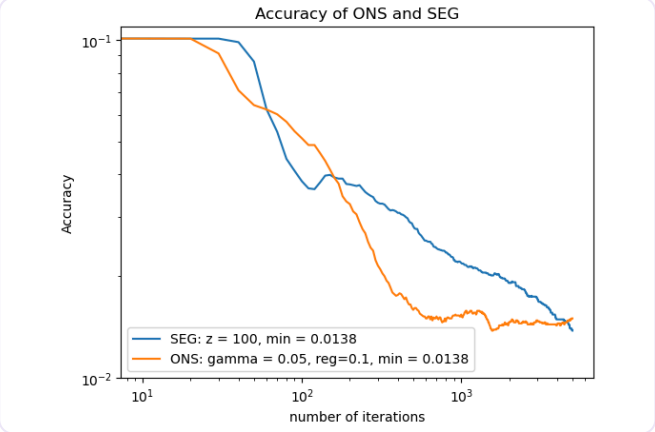


Projected SGD ($z = 100$): the smaller λ, the better; best error 1.5% for $\lambda = 0.05$.

Mirror descent family and second-order method



AdaGrad for several radii z: as for mirror descent and EG±, $z = 100$ works best.



Online Newton Step vs EG±: both reach 1.38% error; ONS needs fewer iterations.

Computational cost

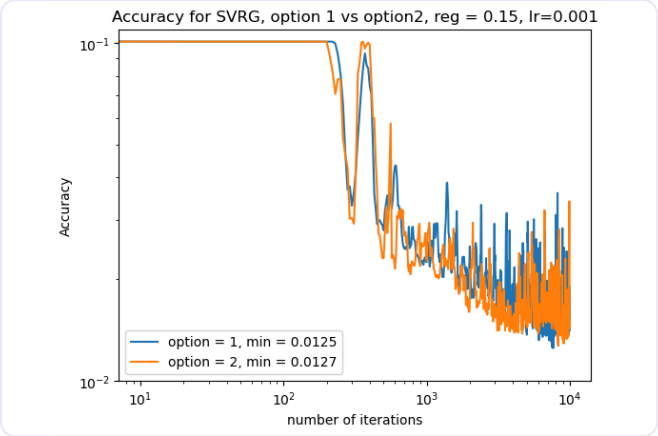
Comparison	Measured times	Ratio
Projected GD vs Projected SGD (10,000 iterations)	190 s vs 0.09 s	SGD ≈ 2,100× faster per iteration
ONS vs EG± (1,000 iterations)	16 s vs 0.03 s	EG± ≈ 460× faster (ONS inverts a 785×785 matrix)
SVRG vs Projected SGD (10,000 iterations, $m = 100$)	4.0 s vs 0.25 s	SGD 16× faster
SVRG vs Projected GD (500 iterations)	0.23 s vs 18.4 s	SVRG 80× faster

5 Reproducing SVRG (Johnson & Zhang, NeurIPS 2013)

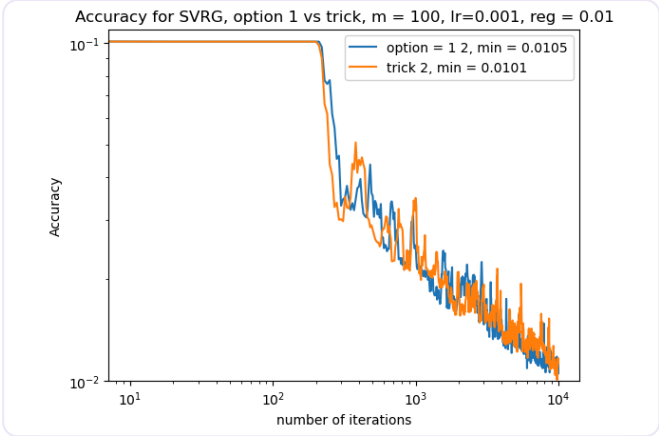
Motivation. The variance of stochastic gradients forces SGD to use a decaying step size, which limits it to a sub-linear $O(1/t)$ rate. SVRG keeps a snapshot $\tilde{w}$ and its full gradient $\mu = \nabla f(\tilde{w})$, refreshed every m iterations, and uses the unbiased, lower-variance estimator $\nabla f_i(x_t) - \nabla f_i(\tilde{w}) + \mu$, which allows a constant step.

Guarantee. If f is α-strongly convex and each f_i is convex and β-smooth, and m is large enough so that $\rho = \frac{1}{\alpha\eta(1-2\beta\eta)m} + \frac{2\beta\eta}{1-2\beta\eta} < 1$, then $\mathbb{E}\, f(\tilde{w}_s) - f(w_*) \leq \rho^s \left[f(\tilde{w}_0) - f(w_*) \right]$: a **linear** convergence rate.

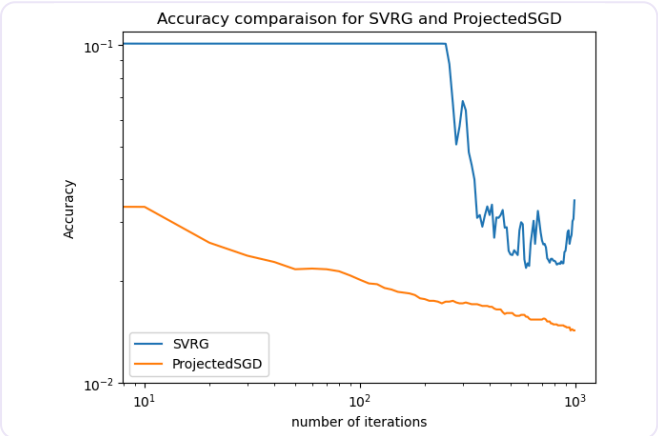
Variant. We also tested a small modification of option I: the full gradient computed at each snapshot is used immediately for the update instead of being discarded.



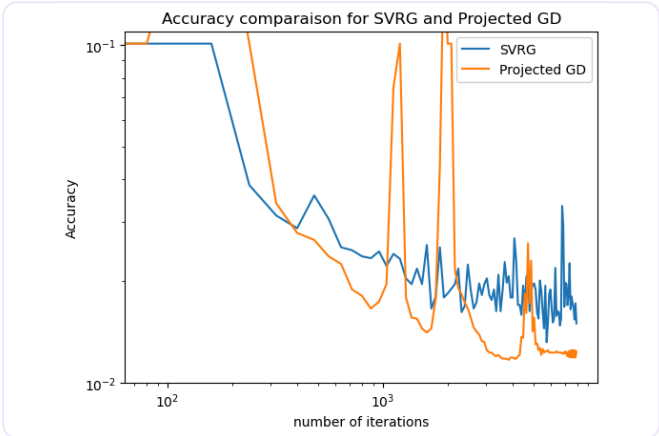
SVRG option I (snapshot = last iterate) vs option II (random iterate of the last epoch).



Option I vs our variant: slightly faster decrease and a slightly lower minimum.



SVRG vs Projected SGD at equal computation time: projected SGD stays ahead.



SVRG vs Projected GD at equal computation time: very similar error.

6 Conclusions

- **Stochastic methods win** at equal budget: projected SGD offers the best trade-off between cost and error on this problem.
- **Constraints help**: projecting on an ℓ_1 ball stabilizes SGD and reduces the noise of the iterates.
- **Second-order information** (ONS) speeds up convergence per iteration, but its $O(d^2)$ memory and matrix inversions make it far slower in wall-clock time.
- **SVRG** does not beat projected SGD here, which is consistent with theory: the hinge loss is **not smooth**, while SVRG's linear rate assumes β-smooth f_i. A smoothed hinge or a logistic loss would be a better setting to observe its advantage.

> The figures come from the original study. In the reusable package of the repository, the SVRG estimator follows the paper exactly ($\nabla f_i(x) - \nabla f_i(\tilde{w}) + \nabla f(\tilde{w})$) and gradients are vectorized.